COMP0086: Unsupervised Learning - Lecture Notes

Comprehensive Course Notes

Contents

Introduction

These lecture notes cover the fundamentals and advanced techniques in unsupervised learning. The course begins with essential mathematical foundations, develops core probabilistic and statistical concepts, and concludes with modern inference methods and graphical models.

The notes are organized into four conceptual parts:

1. **Part I - Foundations**: Linear algebra and probabilistic foundations essential for understanding all subsequent material
2. **Part II - Probabilistic Models**: Core unsupervised learning models including dimensionality reduction, mixture models, and the EM algorithm
3. **Part III - Sequential Models**: Methods for sequences and time series, including Markov chains, HMMs, and state space models
4. **Part IV - Advanced Methods**: Modern sampling, inference, and graphical model techniques

Throughout, we emphasize connections between different frameworks and develop both theoretical understanding and practical intuition for each method.

1 Exponential Family Distributions

The exponential family is a fundamental framework in probabilistic modeling that unifies many standard distributions and simplifies inference algorithms. Any parametric probability distribution that satisfies certain regularity conditions can be written in exponential family form, which reveals underlying structure and enables efficient algorithms.

1.1 Definition and Structure

Definition 1.1 (Exponential Family)**.** A family of probability distributions belongs to the exponential family if it can be written in the form:

$$p(x \mid \boldsymbol{\theta}) = f(x)g(\boldsymbol{\theta})e^{\boldsymbol{\phi}(\boldsymbol{\theta})^T T(x)}$$

where:

- $T(x)$ is the **sufficient statistic** of the data x
- $\boldsymbol{\phi}(\boldsymbol{\theta})$ is the **natural parameter vector**
- $g(\boldsymbol{\theta})$ is the **normalizing constant** that ensures the distribution integrates to 1
- $f(x)$ is the **base measure**, which captures any dependence on the data independent of parameters

The exponential family is a powerful framework because it generalizes many standard distributions and simplifies inference tasks.

1.2 Alternative Formulation with Log-Partition Function

The exponential family can equivalently be written as:

$$p(x \mid \boldsymbol{\theta}) = f(x)e^{\boldsymbol{\phi}(\boldsymbol{\theta})^T T(x) - A(\boldsymbol{\theta})}$$

where $A(\boldsymbol{\theta})$ is called the **log-partition function**.

The relationship between the two formulations is:

$$g(\boldsymbol{\theta}) = e^{-A(\boldsymbol{\theta})}$$

This means $A(\boldsymbol{\theta}) = -\log g(\boldsymbol{\theta})$. The log-partition function is crucial because:

$$\frac{\partial}{\partial \boldsymbol{\theta}} A(\boldsymbol{\theta}) = E[T(x)]$$

Taking the derivative of the log-normalizer gives us the expected value of the sufficient statistic.

2 Common Exponential Family Distributions

2.1 Gaussian Distribution

Example 2.1 (Univariate Gaussian)**.**

$$p(x \mid \mu, \Sigma) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right)$$

Sufficient statistics are the first and second moments of the data, and the natural parameters relate to the mean and precision (inverse covariance).

2.2 Gamma Distribution

Example 2.2 (Gamma Distribution)**.** The Gamma distribution can be expressed in exponential family form with sufficient statistics $(x, \log x)$.

2.3 Beta Distribution

Definition 2.1 (Beta Distribution)**.**

$$p(x \mid \alpha, \beta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1}(1-x)^{\beta-1}$$

for $x \in [0, 1]$, where $\alpha, \beta > 0$. This can be written in exponential family form with sufficient statistics $(\log x, \log(1-x))$.

3 Sufficient Statistics and Their Significance

The key insight of the exponential family is that data x only affects the likelihood through its sufficient statistics $T(x)$. This means:

- All information about $\boldsymbol{\theta}$ is captured in $T(x)$
- We can reduce high-dimensional data to low-dimensional summaries without loss of information
- Two datasets with the same sufficient statistics provide identical evidence for inference about $\boldsymbol{\theta}$

4 Bayesian Inference: MAP and Model Selection

Bayesian inference provides a principled framework for learning from data by combining prior knowledge with observed evidence. Two central tasks are point estimation (finding the best single parameter estimate) and model comparison (determining which model best explains the data). This section develops these concepts and introduces conjugate priors, which enable tractable inference in many practical settings.

4.1 Maximum A Posteriori Estimation

Definition 4.1 (Maximum A Posteriori (MAP))**.** The MAP estimate finds the mode of the posterior distribution:

$$\boldsymbol{\theta}^{\text{MAP}} = \arg\max_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid D) = \arg\max_{\boldsymbol{\theta}} p(D \mid \boldsymbol{\theta}) p(\boldsymbol{\theta})$$

4.2 Model Selection with Bayes' Rule

Definition 4.2 (Model Selection)**.** When comparing multiple models $\mathcal{M}_1, \ldots, \mathcal{M}_k$, we compute the posterior probability of each model given data D:

$$P(\mathcal{M}_i \mid D, \mathcal{M}_i) = \frac{P(D \mid \mathcal{M}_i) P(\mathcal{M}_i)}{P(D)}$$

where $P(D \mid \mathcal{M}_i)$ is the marginal likelihood under model i:

$$P(D \mid \mathcal{M}_i) = \int P(D \mid \boldsymbol{\theta}_i, \mathcal{M}_i) P(\boldsymbol{\theta}_i \mid \mathcal{M}_i) d\boldsymbol{\theta}_i$$

Remark 4.1. Posterior model probability

$$P(\mathcal{M}_i \mid D, \mathcal{M}_i) = \frac{P(D \mid \mathcal{M}_i) P(\mathcal{M}_i)}{\sum_j P(D \mid \mathcal{M}_j) P(\mathcal{M}_j)}$$

5 Conjugate Priors

5.1 The Concept of Conjugacy

Definition 5.1 (Conjugate Prior)**.** A prior distribution $p(\boldsymbol{\theta})$ is **conjugate** to the likelihood $p(D \mid \boldsymbol{\theta})$ if the posterior distribution $p(\boldsymbol{\theta} \mid D)$ is in the same family as the prior. Conjugate priors are particularly useful because they lead to closed-form posterior updates.

The choice of conjugate prior is natural for exponential family likelihoods. The general form of a conjugate prior for exponential family distributions is:

$$p(\boldsymbol{\phi}) = F(\tau, \nu) g(\boldsymbol{\phi})^{\tau} e^{\nu \boldsymbol{\phi}^T T_0}$$

where:

- $F(\tau, \nu)$ is the normalizing constant
- $\tau > 0$ is the concentration or scale of the prior
- $\nu \in \mathbb{R}_{>0}$ concentrates the prior around a mean
- T_0 is a pseudo-data contribution

The posterior given a conjugate prior is:

$$p(\boldsymbol{\phi} \mid D, \mathcal{M}_i) = F(T + \sum_{j=1}^{n} T_j(x_i), \nu + n)$$

where the posterior parameters are updated by adding data contributions to prior hyperparameters.

5.2 Conjugate Prior: Bernoulli-Beta Example

Example 5.1 (Bernoulli Likelihood with Beta Conjugate Prior)**.** Consider a Bernoulli likelihood for binary observations. The natural conjugate prior is the Beta distribution.

Prior: $p(\theta) = \text{Beta}(\tau + 1, \nu - \tau + 1)$

Posterior: Given data from n observations with counts, the posterior is:

$$p(\theta \mid D) \propto \text{Beta}\left(\tau + \sum_{i=1}^{n} x_i + 1, \nu - \tau + n - \sum_{i=1}^{n} x_i + 1\right)$$

5.3 Posterior Given Conjugate Prior

When using a conjugate prior with exponential family likelihood, the posterior remains in the same family. For example:

Example 5.2 (Bernoulli-Beta Conjugacy)**. Model:** Bernoulli observations with Beta conjugate prior
Prior: $\text{Beta}(\tau + 1, \nu - \tau + 1)$
Posterior:

$$p(\theta \mid D) = \text{Beta}\left(\tau + \sum_{i=1}^{n} x_i + 1, \nu - \tau + n - \sum_{i=1}^{n} x_i + 1\right)$$

The posterior is also Beta distributed, with hyperparameters updated based on the observed data.

6 Information-Theoretic Quantities

6.1 Fisher Information Matrix

The **Fisher Score** (or Fisher Information Matrix) quantifies the curvature of the log-likelihood and measures the amount of information the data carries about the parameters:

$$\nabla_{\boldsymbol{\theta}} \log p(x \mid \boldsymbol{\theta})$$

The Fisher Information Matrix is defined as:

$$\mathcal{I}(\boldsymbol{\theta}) = E\left[\left(\nabla_{\boldsymbol{\theta}} \log p(x \mid \boldsymbol{\theta})\right)\left(\nabla_{\boldsymbol{\theta}} \log p(x \mid \boldsymbol{\theta})\right)^T\right]$$

An important property: $\mathbb{E}[\nabla_{\boldsymbol{\theta}} \log p(x \mid \boldsymbol{\theta})] = 0$ (the score has zero mean).
The Fisher Information is crucial for:

- Determining the asymptotic variance of the MLE: $\text{Var}(\hat{\boldsymbol{\theta}}_{\text{MLE}}) \approx \mathcal{I}(\boldsymbol{\theta})^{-1}/n$
- Computing the Cramér–Rao lower bound on estimator variance

7 Linear Algebra: Matrix Properties and Orthonormal Bases

Linear algebra provides essential tools for understanding covariance structure, data transformations, and optimization. Many unsupervised learning algorithms rely on eigenvalue decompositions and orthogonal transformations to find natural coordinate systems for high-dimensional data. In this section, we review key results that form the backbone of algorithms like PCA and spectral clustering.

7.1 Eigenvalues of Symmetric Matrices

Theorem 7.1 (Spectral Properties of Symmetric Matrices)**.** Symmetric real matrices have particularly nice properties:

1. All eigenvalues are real
2. Eigenvectors corresponding to distinct eigenvalues are orthogonal
3. A symmetric matrix of dimension $d \times d$ has exactly d eigenvalues (counting multiplicities)

These properties are fundamental for many algorithms in unsupervised learning, particularly in principal component analysis (PCA) and spectral clustering.

7.2 Transforming Between Orthonormal Bases

Definition 7.1 (Orthonormal Basis Transformation)**.** Let $\mathcal{U} = \{u_1, \ldots, u_d\}$ and $\mathcal{V} = \{v_1, \ldots, v_d\}$ be two orthonormal bases (ONBs) in $\mathbb{R}^d$.

We can express the change of basis using an orthogonal transformation matrix:

$$O = \begin{bmatrix} \langle u_1, v_1 \rangle & \langle u_1, v_2 \rangle & \cdots \\ \langle u_2, v_1 \rangle & \langle u_2, v_2 \rangle & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

with the property that $O^T O = I$ (orthogonal matrix). For coordinates, if y are coordinates in the $\mathcal{V}$ basis and x are coordinates in the $\mathcal{U}$ basis, then:

$$x = Oy$$

The transformation has the form:

$$w_j = \frac{1}{\sqrt{n}} \sum_{i=1}^{|V|} O_{ji} v_i$$

This linear transformation preserves distances and angles, making it fundamental for algorithms like PCA where we want to find new coordinate systems that capture data variance.

8 Quadratic Forms and Matrix Properties

8.1 Quadratic Forms

Definition 8.1 (Quadratic Form)**.** For a symmetric matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ and vector $\mathbf{x} \in \mathbb{R}^n$, the quadratic form is:

$$\mathbf{x}^T \mathbf{A} \mathbf{x}$$

The quadratic form tells us about the curvature properties of the matrix:

Definition 8.2 (Positive/Negative Definite Matrices)**.** A symmetric matrix $\mathbf{A}$ is:

- **Positive definite:** $\mathbf{x}^T \mathbf{A} \mathbf{x} > 0$ for all $\mathbf{x} \neq 0$
- **Positive semi-definite:** $\mathbf{x}^T \mathbf{A} \mathbf{x} \geq 0$ for all $\mathbf{x}$
- **Negative definite:** $\mathbf{x}^T \mathbf{A} \mathbf{x} < 0$ for all $\mathbf{x} \neq 0$

Quadratic forms with positive/negative definite matrices define **elliptic quadratic forms**.

8.2 Eigenvalue Decomposition of the Covariance Matrix

Theorem 8.1 (Eigenvalue Decomposition)**.** For a symmetric matrix $\mathbf{A}$, the eigenvalue decomposition is:

$$\mathbf{A} = \mathbf{P} \mathbf{D} \mathbf{P}^T$$

where:

- $\mathbf{P}$ is the **eigenvector matrix** (columns are eigenvectors)
- $\mathbf{D}$ is the **diagonal matrix of eigenvalues**
- $\mathbf{P}$ is orthogonal: $\mathbf{P}^T \mathbf{P} = \mathbf{I}$

This transformation expresses the matrix in terms of its eigenvector basis. The eigenvectors form a new basis, and the eigenvalues tell us the scaling factor along each basis direction.

Remark 8.1. Geometric Interpretation The eigenvector matrix $\mathbf{P}$ serves as a transformation from the original basis to a new eigenbasis. In this new coordinate system, the matrix $\mathbf{A}$ becomes diagonal, with the eigenvalues on the diagonal. This is the key insight behind Principal Component Analysis (PCA).

8.3 Multivariate Gaussians in Coordinate Systems

Theorem 8.2 (Multivariate Gaussian Representation)**.** A multivariate Gaussian with arbitrary covariance matrix can be represented as independent (uncorrelated) Gaussians in an appropriate coordinate system. Specifically, if $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, then in the eigenvector basis of $\boldsymbol{\Sigma}$, the components are independent Gaussians.

Mathematically:

$$\mathbf{z} = \mathbf{P}^T (\mathbf{x} - \boldsymbol{\mu})$$

where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{D})$ and $\mathbf{D}$ is diagonal.

This property is fundamental for understanding covariance structure and for algorithms like PCA.

9 Matrix Calculus

9.1 Basic Matrix Derivatives

Computing derivatives with respect to matrices is essential for optimization in machine learning. Some key results:

Proposition 9.1 (Matrix Derivatives)**.**

$$\frac{\partial}{\partial \mathbf{A}} \mathrm{Tr}(\mathbf{G}\mathbf{A}^T\mathbf{B}) = \mathbf{B}\mathbf{G}^T \tag{1}$$

$$\frac{\partial}{\partial \mathbf{A}} \mathrm{Tr}(\mathbf{G}\mathbf{A}^T\mathbf{B}\mathbf{A}\mathbf{C}^T) = \mathbf{B}\mathbf{A}\mathbf{C} + \mathbf{B}^T\mathbf{A}\mathbf{C}^T \tag{2}$$

$$\frac{\partial}{\partial \mathbf{A}} \log|\mathbf{A}| = (\mathbf{A}^{-1})^T = \mathbf{A}^{-T} \tag{3}$$

The last result is particularly important for probabilistic models where the likelihood involves the determinant of the covariance matrix.

10 Linear Gaussian Models

10.1 Linear Regression with Gaussian Noise

Definition 10.1 (Linear Gaussian Model)**.** Consider a linear model with Gaussian noise:

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \nu I)$$

where:

- $\mathbf{W}$ is the weight matrix to be learned
- ν is the noise variance
- Observations are $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$

The optimal weights (maximum likelihood estimate) are:

$$\hat{\mathbf{W}}_{ML} = \sum_{i=1}^{n} \mathbf{y}_i \mathbf{x}_i^T \left(\sum_{i=1}^{n} \mathbf{x}_i \mathbf{x}_i^T \right)^{-1}$$

10.2 MAP Estimation with Gaussian Prior

When we place a Gaussian prior on the weights:

$$p(\mathbf{W} \mid \mathbf{A}) = \mathcal{N}(\mathbf{0}, \mathbf{A}^{-1})$$

The MAP estimate becomes:

$$\mathbf{W}^{\mathrm{MAP}} = \left(\mathbf{A} + \sum_{i=1}^{n} \mathbf{x}_i \mathbf{x}_i^T \right)^{-1} \sum_{i=1}^{n} \mathbf{y}_i \mathbf{x}_i^T$$

The prior precision matrix $\mathbf{A}$ acts as a regularizer, shrinking the weights toward zero.

11 Dimensionality Reduction

High-dimensional data often lies near low-dimensional manifolds, and many machine learning tasks become easier in these lower-dimensional representations. This section develops probabilistic approaches to dimensionality reduction, where Probabilistic PCA extends classical PCA with a full probabilistic model that quantifies uncertainty. By learning low-dimensional representations, these methods reduce computational burden, improve generalization, and often reveal interpretable structure in the data.

11.1 Probabilistic Principal Component Analysis (PPCA)

Definition 11.1 (Probabilistic PCA Model)**.** Probabilistic PCA extends standard PCA with a probabilistic framework:

$$z \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad \text{(latent variables)}$$

$$p(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mathbf{\Lambda z}, \nu \mathbf{I})$$

$$p(\mathbf{x}) = \int p(\mathbf{z}) p(\mathbf{x} \mid \mathbf{z}) d\mathbf{z} = \mathcal{N}(\mathbf{0}, \mathbf{\Lambda \Lambda}^T + \nu \mathbf{I})$$

where:

- $\mathbf{z} \in \mathbb{R}^K$ is a latent variable with $K < D$ (dimensionality reduction)
- $\mathbf{\Lambda} \in \mathbb{R}^{D \times K}$ is the loading matrix
- ν is the noise variance in the observation space

Remark 11.1 (Variance Decomposition)**.** The variance of the data can be decomposed as:

$$\text{Var}(\mathbf{X}) = \mathbb{E}[\text{Var}(\mathbf{X} \mid \mathbf{Z})] + \text{Var}(\mathbb{E}[\mathbf{X} \mid \mathbf{Z}])$$

This shows that PPCA captures variance in two parts: the variance explained by the latent factors and the residual noise variance.

Remark 11.2 (Dimensionality Limitation)**.** If $\nu \to 0$, the PPCA model can only capture K dimensions of variance. The latent dimension K determines the maximum rank of variance that can be represented.

11.2 Principal Component Analysis (PCA) Algorithm

Definition 11.2 (PCA Objective)**.** PCA seeks to find K orthonormal directions that maximize the variance in the data:

$$\lambda_K = \arg\max_{\mathbf{v}} \sum_{i=1}^{n} \|\mathbf{X}_i \cdot \mathbf{v}\|^2 \quad \text{subject to} \quad \|\mathbf{v}\|^2 = 1$$

Proposition 11.1 (PCA Algorithm)**.** To find the top K principal components:

1. Compute the sample covariance matrix:

$$\hat{\mathbf{\Sigma}} = \frac{1}{n} \sum_{i=1}^{n} \mathbf{X}_i \mathbf{X}_i^T$$

2. Perform eigenvalue decomposition: $\hat{\mathbf{\Sigma}} = \mathbf{U}\mathbf{\Lambda}^2\mathbf{U}^T$

3. Select the top K eigenvectors with largest eigenvalues to form:

$$\mathbf{U}_K = [\mathbf{u}_1, \ldots, \mathbf{u}_K]$$

4. Project the data to the lower-dimensional space:

$$\mathbf{Z}_i = \mathbf{U}_K^T \mathbf{X}_i$$

5. The reconstruction is:

$$\hat{\mathbf{X}}_i = \mathbf{U}_K \mathbf{U}_K^T \mathbf{X}_i$$

which is an orthogonal projection onto the K-dimensional subspace.

Remark 11.3 (Key Differences: PCA vs PPCA)**.**

- In **PCA**, the noise is assumed to be isotropic and orthogonal to all principal components.
- In **PPCA**, the noise is explicitly modeled in the observation space and is taken into account when determining the principal subspace.
- When $\nu \to 0$, PPCA reduces to standard PCA.

11.3 Posterior Inference in PPCA

Given an observation $\mathbf{x}_i$, we can infer the latent variables using:

$$\mathbb{E}[Z \mid \mathbf{x}_i] = \mathbf{M}^{-1}\mathbf{\Lambda}^T\nu^{-1}\mathbf{x}_i$$

where $\mathbf{M} = \mathbf{\Lambda}^T\nu^{-1}\mathbf{\Lambda} + \mathbf{I}$ is the posterior precision.
The posterior variance is:

$$\mathrm{Cov}(Z \mid \mathbf{x}_i) = \mathbf{M}^{-1}$$

As the noise variance $\nu \to 0$, the PPCA framework transitions toward standard PCA.

12 Mixture Models

Mixture models are a flexible class of probabilistic models that assume data comes from one of M different components. By treating the component assignment as a latent variable, we can use the EM algorithm to learn both the component parameters and the mixing proportions. K-means clustering emerges as a limiting case where all uncertainty is removed. Mixture models show how probabilistic modeling naturally leads to useful algorithms for unsupervised learning.

12.1 Mixture Model Likelihood

Definition 12.1 (Mixture Model)**.** A mixture model models the data as coming from one of M components:

$$p(\mathbf{x}_i) = \sum_{m=1}^{M} P(c_i = m)p(\mathbf{x}_i \mid c_i = m) = \sum_{m=1}^{M} \pi_m p(\mathbf{x}_i \mid \boldsymbol{\theta}_m)$$

where:

- $\pi_m = P(c_i = m)$ are the mixture weights (proportions)
- $\boldsymbol{\theta}_m$ are the parameters of component m
- c_i is the component indicator for data point i

12.2 Posterior Responsibility

Given a data point $\mathbf{x}_i$, we can compute the probability that it belongs to component m using Bayes' rule:

Definition 12.2 (Responsibility)**.**

$$P(c_i = m \mid \mathbf{x}_i) = \frac{\pi_m p(\mathbf{x}_i \mid \boldsymbol{\theta}_m)}{\sum_{m'=1}^{M} \pi_{m'} p(\mathbf{x}_i \mid \boldsymbol{\theta}_{m'})}$$

This quantity is called the **responsibility** of component m for data point i, often denoted ρ_{im}.

12.3 Maximum Likelihood Estimation in Mixture Models

Definition 12.3 (Log-Likelihood for Mixture Models)**.** The log-likelihood of the data is:

$$\mathcal{L} = \sum_{i=1}^{n} \log \sum_{m=1}^{M} \pi_m p(\mathbf{x}_i \mid \boldsymbol{\theta}_m)$$

Taking derivatives and setting them to zero gives the MLE updates:

$$\pi_m \propto \sum_{i=1}^{n} P(c_i = m \mid \mathbf{x}_i) \tag{4}$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_m} = \sum_{i=1}^{n} P(c_i = m \mid \mathbf{x}_i) \frac{\partial \log p(\mathbf{x}_i \mid \boldsymbol{\theta}_m)}{\partial \boldsymbol{\theta}_m} \tag{5}$$

These updates are the basis for the Expectation-Maximization (EM) algorithm, which alternates between computing responsibilities (E-step) and updating parameters (M-step).

12.4 K-Means Clustering Algorithm

Definition 12.4 (K-Means Algorithm)**.** K-Means is a special case of the mixture model where we assume the responsibility is deterministic (hard assignment) and the noise variance goes to zero ($\sigma \to 0$). In this limit:

$$\rho_{im} \to \delta(m, \arg\min_m \|\mathbf{x}_i - \boldsymbol{\mu}_m\|^2)$$

The K-Means algorithm alternates between two steps:

1. **Assignment Step:** Assign each point to its closest cluster mean:

$$c_i = \arg\min_m \|\mathbf{x}_i - \boldsymbol{\mu}_m\|^2$$

2. **Update Step:** Recalculate the cluster means as the centroid of assigned points:

$$\boldsymbol{\mu}_m = \frac{\sum_{i:c_i=m} \mathbf{x}_i}{\sum_{i:c_i=m} 1}$$

Note: K-Means has no learning rate. At each iteration, the parameter update is the exact optimal solution for the current assignment. The algorithm is guaranteed to converge (though possibly to a local optimum).

13 Information Theory

Information theory quantifies uncertainty and provides a principled framework for understanding data compression, divergence measures, and model comparison. The key insight is that information and probability are deeply connected—uncertainty about a random variable can be measured precisely using Shannon entropy, and differences between probability distributions can be quantified using divergence measures. These concepts form the theoretical foundation for many machine learning algorithms.

13.1 Information and Entropy

Definition 13.1 (Information Content)**.** When we observe a data point x sampled from a distribution P, the information (or surprise) is:

$$I(x) := \log \frac{1}{p(x)} = -\log P(x)$$

This quantity is **additive** in the sense that observing independent events yields additive information. Data with low probability provides more information (high surprise), while high-probability data provides less information.

Definition 13.2 (Shannon Entropy)**.** The Shannon entropy of a random variable X is the expected value of the information:

$$H[X] := \mathbb{E}_P[-\log p(x)] = -\sum_x p(x) \log p(x)$$

Entropy measures the average surprise or uncertainty in the distribution.

Theorem 13.1 (Properties of Shannon Entropy)**.** 1. **Non-negativity:** $H[X] \geq 0$, with equality if and only if the distribution is deterministic ($p(x_i) = 1$ for some x_i).

2. **Maximum entropy:** For a finite distribution with d elements, the maximum entropy is achieved by the uniform distribution:

$$\max_P H[X] = \log d \quad \text{(achieved by uniform distribution)}$$

3. **Concavity:** Entropy is a concave function of the distribution p.

13.2 Axioms for Information Measures

Any reasonable measure of information should satisfy certain fundamental axioms. Shannon showed that these axioms uniquely characterize the entropy function.

Theorem 13.2 (Shannon's Axioms)**.** A function H on distributions P over space X is a valid measure of information if and only if it satisfies:

1. **Additivity for independent variables:** For independent X and Y:

$$H(X, Y) = H(X) + H(Y \mid X)$$

2. **Continuity:** $H(P)$ is a continuous function of the probability distribution.

3. **Monotonicity in dimension:** As the number of equally likely outcomes increases, the uncertainty increases:

$$H(U(d)) < H(U(d+1))$$

where $U(d)$ is the uniform distribution over d elements.

Any function satisfying these three axioms has the form:

$$H(P) = c \cdot H_{\text{Shannon}}(P)$$

for some positive constant c (typically $c = 1$ with logarithm base 2 for bits).

13.3 Huffman Coding

Definition 13.3 (Huffman Coding)**.** Huffman coding is a lossless compression algorithm that assigns shorter binary codes to more frequent symbols and longer codes to less frequent symbols. This minimizes the expected code length.

Algorithm:

1. Count the frequency of each symbol
2. Build a binary tree by repeatedly merging the two symbols with lowest frequency
3. Assign codes: left branch = 0, right branch = 1
4. Read from root to leaf to get each symbol's code

Example: For the string "ABRACADABRA":

- Frequencies: A=5, B=2, R=2, C=1, D=1
- Huffman codes: A=0, B=10, R=110, C=1110, D=1111
- This achieves compression close to the Shannon entropy bound

13.4 Source Coding Theorem

Theorem 13.3 (Shannon's Source Coding Theorem)**.** For i.i.d. samples $X_1, X_2, \ldots, X_n$ from distribution P, and an encoder $\mathcal{C}$ that maps sequences to binary strings:

$$H(P) \leq \mathbb{E}[\text{length}(\mathcal{C}(X_1, \ldots, X_n))]/n \leq H(P) + \varepsilon$$

for sufficiently large n.

This means:

- **Lower bound:** The entropy $H(P)$ is the fundamental limit on lossless compression. No lossless code can achieve an average code length less than $H(P)$ bits per symbol.
- **Achievability:** For large enough n, there exists a code (like Huffman coding) that gets arbitrarily close to the entropy limit.
- **Key insight:** The minimum average code length depends on the distribution P only.

14 Divergence Measures and Model Comparison

Beyond Shannon entropy, we need tools to compare probability distributions, measure how far a learned model is from the true distribution, and select between competing models. Divergence measures like KL divergence and mutual information formalize these concepts. These tools are essential for model selection, for understanding why maximum likelihood estimation works, and for variational inference—approximate inference methods that minimize divergences.

14.1 Kullback-Leibler Divergence

Definition 14.1 (Kullback-Leibler (KL) Divergence)**.** The KL divergence measures the difference between two probability distributions P and Q over the same space:

$$\text{KL}(P\|Q) := \mathbb{E}_P\left[\log \frac{p(x)}{q(x)}\right] = -\sum_x p(x) \log q(x) + \sum_x p(x) \log p(x)$$

$$= \sum_x p(x) \log \frac{p(x)}{q(x)}$$

The KL divergence can be interpreted as:

- The expected information gained by switching from Q to P
- The cross-entropy minus the entropy: $\text{KL}(P\|Q) = H(P, Q) - H(P)$

Theorem 14.1 (Properties of KL Divergence)**.**

1. **Non-negativity:** $\text{KL}(P\|Q) \geq 0$, with equality if and only if $P = Q$.
2. **Asymmetry:** In general, $\text{KL}(P\|Q) \neq \text{KL}(Q\|P)$.
3. **Convexity:** KL divergence is convex in both its arguments.

Remark 14.1 (Interpretation of Asymmetry)**.** The asymmetry of KL divergence is important:

- $\mathrm{KL}(P\|Q)$ is small when Q is concentrated where P is concentrated (mode-seeking behavior)
- $\mathrm{KL}(Q\|P)$ is small when Q has support where P has support (mass-covering behavior)

This difference is important in variational inference, where we often minimize $\mathrm{KL}(Q\|P)$ to find a simpler distribution Q that approximates the true posterior P.

14.2 Cross-Entropy

Definition 14.2 (Cross-Entropy)**.** The cross-entropy between distributions P and Q is:

$$H(P, Q) := -\sum_x p(x) \log q(x)$$

This measures the expected code length if we use codes designed for distribution Q when the true distribution is P. Alternatively, it measures the average surprisal under the wrong distribution.

The cross-entropy can be decomposed as:

$$H(P, Q) = H(P) + \mathrm{KL}(P \| Q)$$

where $H(P)$ is the entropy of P (the minimum possible code length). Thus, KL divergence measures the additional cost of using the wrong distribution.

14.3 Mutual Information

Definition 14.3 (Mutual Information)**.** The mutual information between random variables X and Y measures their statistical dependence:

$$I[X, Y] := \mathrm{KL}(P(X, Y) \| P(X)P(Y)) = \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$

Properties:

- $I[X, Y] \geq 0$, with equality if and only if $X \perp Y$ (independence)
- $I[X, Y] = I[Y, X]$ (symmetry)
- Mutual information measures how much knowing Y reduces uncertainty about X

14.4 Differential Entropy and Continuous Case

Definition 14.4 (Differential Entropy)**.** For continuous random variables, entropy is defined as:

$$H[X] := \int_X p(x) \log p(x)\, dx$$

Important distinction: Unlike discrete entropy, differential entropy can be negative. This reflects the fact that continuous distributions can have arbitrarily high precision, reducing uncertainty below one bit.

The properties of KL divergence and mutual information extend to the continuous case in a straightforward manner.

15 Maximum Entropy and Maximum Likelihood

15.1 Exponential Family as Maximum Entropy Model

Theorem 15.1 (Maximum Entropy for Exponential Families)**.** Among all distributions satisfying the constraint $\mathbb{E}[T(X)] = t$, the exponential family distribution with natural parameters $\boldsymbol{\phi}$ chosen such that the constraint is satisfied has maximum entropy.

Conversely, if we want to model a distribution with maximum entropy subject to constraints on expected values of certain features, the exponential family is the natural choice.

Remark 15.1. Practical Implication When building probabilistic models (especially in Bayesian settings), if we know only the expected value of some sufficient statistics but want to avoid over-committing to a specific distribution, using the exponential family is a principled choice that makes minimal assumptions.

15.2 Maximum Likelihood as KL Minimization

Theorem 15.2 (ML Minimizes KL Divergence)**.** Maximum likelihood estimation can be interpreted as minimizing the KL divergence between the empirical distribution and the model:

$$\boldsymbol{\theta}^{\mathrm{ML}} = \arg\min_{\boldsymbol{\theta}} \mathrm{KL}(\hat{P}_{\mathrm{emp}} \| p(\cdot \mid \boldsymbol{\theta}))$$

where $\hat{P}_{\mathrm{emp}}$ is the empirical distribution on the data.

16 Variational Inference and the EM Algorithm

Many unsupervised learning problems involve latent (unobserved) variables that explain the observed data. The Free Energy and the Expectation-Maximization (EM) algorithm provide powerful tools for learning from incomplete data. EM alternates between inferring the latent variables (E-step) and optimizing parameters (M-step), making it one of the most widely used algorithms in unsupervised learning. This section develops the theoretical foundation and shows how EM unifies clustering, dimensionality reduction, and mixture modeling.

16.1 Free Energy and Variational Lower Bound

Definition 16.1 (Free Energy)**.** For a model with hidden variables Z and observations X, the free energy is:

$$\mathcal{F}(q, \boldsymbol{\theta}) = \mathbb{E}_q[\log p(X, Z \mid \boldsymbol{\theta})] - H[q(Z)]$$

where $q(Z)$ is a distribution over the latent variables.

This can be rewritten as:

$$\mathcal{F}(q, \boldsymbol{\theta}) = \log p(X \mid \boldsymbol{\theta}) - \mathrm{KL}(q(Z) \| p(Z \mid X, \boldsymbol{\theta}))$$

Theorem 16.1 (Variational Lower Bound)**.** The free energy forms a lower bound on the log-likelihood:

$$\mathcal{F}(q, \boldsymbol{\theta}) \leq \log p(X \mid \boldsymbol{\theta})$$

with equality when $q(Z) = p(Z \mid X, \boldsymbol{\theta})$ (the true posterior).

This bound is fundamental to variational inference: we optimize the free energy as a proxy for the intractable likelihood.

16.2 Expectation-Maximization Algorithm

Definition 16.2 (EM Algorithm)**.** The Expectation-Maximization algorithm maximizes the free energy by alternating between:

1. **E-step (Expectation):** Optimize the free energy with respect to the posterior distribution $q(Z)$, holding $\boldsymbol{\theta}$ fixed:

$$q^{(k)}(Z) = \arg\max_{q} \mathcal{F}(q, \boldsymbol{\theta}^{(k-1)})$$

   This gives $q^{(k)}(Z) = p(Z \mid X, \boldsymbol{\theta}^{(k-1)})$, the true posterior under current parameters.

2. **M-step (Maximization):** Optimize the free energy with respect to the parameters $\boldsymbol{\theta}$, holding q fixed:

$$\boldsymbol{\theta}^{(k)} = \arg\max_{\boldsymbol{\theta}} \mathcal{F}(q^{(k)}(Z), \boldsymbol{\theta}) = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{n} \sum_{z} q^{(k)}(z_i) \log p(x_i, z_i \mid \boldsymbol{\theta})$$

After convergence, the free energy equals the likelihood.

16.3 EM for K-Means

For K-Means, the EM algorithm simplifies to:

- **E-step:** Assign each point to its nearest cluster center: $c_i = \arg\min_m \|\mathbf{x}_i - \boldsymbol{\mu}_m\|^2$
- **M-step:** Update cluster centers: $\boldsymbol{\mu}_m = \frac{1}{|C_m|} \sum_{i \in C_m} \mathbf{x}_i$

16.4 EM for Exponential Family Models

When the joint distribution $p(Z, X \mid \boldsymbol{\theta})$ belongs to the exponential family:

$$p(Z, X \mid \boldsymbol{\theta}) = f(Z, X) \exp(\boldsymbol{\phi}(\boldsymbol{\theta})^T T(Z, X) - A(\boldsymbol{\theta}))$$

The free energy becomes:

$$\mathcal{F}(q, \boldsymbol{\theta}) = \boldsymbol{\phi}(\boldsymbol{\theta})^T \mathbb{E}_q[T(Z, X)] - A(\boldsymbol{\theta}) + H[q] + \text{const}$$

E-step: Compute the expected sufficient statistics under the posterior:

$$\mathbb{E}_q[T(Z, X)] = \int q(z) f(z) \exp(\boldsymbol{\phi}(\boldsymbol{\theta})^T T(z, x)) T(z, x) dz$$

M-step: Moment matching - set the expected sufficient statistics under the posterior equal to those under the current model. This is accomplished by solving:

$$\frac{\partial \mathcal{F}}{\partial \boldsymbol{\phi}} = \mathbb{E}_q[T(Z, X)] - \mathbb{E}_p[T(Z, X) \mid \boldsymbol{\theta}] = 0$$

Thus, the EM algorithm reduces to computing expected sufficient statistics in both steps, with parameter updates determined by moment matching.

16.5 EM for MAP Estimation with Conjugate Priors

When we have a conjugate prior on the parameters, the EM algorithm can be modified to maximize the posterior instead of the likelihood. If the prior is:

$$p(\boldsymbol{\theta}) = F(\tau, \nu)\frac{\exp(\boldsymbol{\theta}^T \tau\boldsymbol{\zeta})}{Z(\boldsymbol{\theta})^\nu}$$

The MAP-EM algorithm modifies the free energy to include the log prior:

$$\mathcal{F}_{\text{MAP}}(q, \boldsymbol{\theta}) = \mathbb{E}_q[\log p(Z, X \mid \boldsymbol{\theta})] + \log p(\boldsymbol{\theta}) + H[q]$$

The resulting updates incorporate data as if there were $(N + \nu)$ pseudo-observations with expected sufficient statistics adjusted by the prior.

16.6 Gaussian Mixture Model

Definition 16.3 (GMM with EM Updates)**.** For a Gaussian mixture model with M components:

E-step: Compute responsibilities (soft assignments):

$$\rho_{im} = P(c_i = m \mid \mathbf{x}_i) \propto \pi_m \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_m, \sigma_m^2 I)$$

M-step: Update parameters:

$$\boldsymbol{\mu}_m = \frac{\sum_{i=1}^n \rho_{im}\mathbf{x}_i}{\sum_{i=1}^n \rho_{im}}, \quad \sigma_m^2 = \frac{\sum_{i=1}^n \rho_{im}\|\mathbf{x}_i - \boldsymbol{\mu}_m\|^2}{\sum_{i=1}^n \rho_{im}}, \quad \pi_m = \frac{1}{n}\sum_{i=1}^n \rho_{im}$$

16.7 Discrete Latent Variables in Vector Form

Definition 16.4 (One-Hot Encoding for Discrete Latents)**.** For discrete latent variables taking values in $\{1, \ldots, M\}$, we can encode them in one-hot vector form:

$$\mathbf{s}_i = \mathbf{e}_m \quad \Rightarrow \quad S_i = [0, 0, \ldots, 1, \ldots, 0]^T$$

where the 1 is in position m.

For exponential family models with discrete latents, the sufficient statistics and the log-partition function can be organized:

$$\boldsymbol{\Theta} = [\boldsymbol{\theta}_1^T; \ldots; \boldsymbol{\theta}_M^T] \quad (M \times d \text{ matrix})$$

$$\log p(\mathbf{x}, \mathbf{s}) = \sum_i [(\log \boldsymbol{\pi})^T \mathbf{s}_i + \mathbf{s}_i^T \boldsymbol{\Theta} T(\mathbf{x}_i) - \mathbf{s}_i^T \log Z(\boldsymbol{\theta})] + \text{const}$$

This is linear in the discrete latent variables, making the E-step tractable.

17 Temporal Models: State Space and Hidden Markov Models

Many real-world datasets have a temporal or sequential structure where current observations depend on past values. This section develops probabilistic models for sequential data, starting with the general framework of latent variable models over time, then specialized to linear Gaussian state space models and hidden Markov models. These models separate the observed data from latent state variables that capture the essential dynamics, enabling both filtering (online state estimation) and smoothing (offline reconstruction of hidden states).

17.1 General Latent Variable Models

Definition 17.1 (Joint Probability for Latent Variable Models)**.** For a sequence model with latent variables $Z_{1:T}$ and observations $X_{1:T}$:

$$P(Z_{1:T}, X_{1:T}) = P(Z_1)P(X_1 \mid Z_1)\prod_{t=2}^{T} P(Z_t \mid Z_{t-1})P(X_t \mid Z_t)$$

This factorization captures:

- A Markov transition model on latent states
- Observation models that depend only on the current latent state
- Conditional independence structure that makes inference tractable

17.2 Linear Gaussian State Space Models

Definition 17.2 (Linear Gaussian SSM)**.** A linear Gaussian state space model (SSM) consists of:

Initial state: $\mathbf{z}_1 \sim \mathcal{N}(\boldsymbol{\mu}_0, \Sigma_0)$
State dynamics (transition): $\mathbf{z}_{t+1} = \mathbf{A}\mathbf{z}_t + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$
Observation model: $\mathbf{x}_t = \mathbf{C}\mathbf{z}_t + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$
where:

- $\mathbf{A}$ is the state transition matrix
- $\mathbf{C}$ is the observation matrix
- $\mathbf{R}$ and $\mathbf{Q}$ are the process and measurement noise covariances
- $\mathbf{v}_t, \mathbf{w}_t$ are zero-mean, uncorrelated noise vectors

Linear Gaussian SSMs are particularly useful because inference (Kalman filtering) has closed-form solutions.

17.3 Hidden Markov Models

Definition 17.3 (Hidden Markov Model (HMM))**.** An HMM models sequential data with discrete hidden states $S_{1:T}$ and observations $X_{1:T}$:

$$P(S_{1:T}, X_{1:T}) = P(S_1)P(X_1 \mid S_1)\prod_{t=2}^{T} P(S_{t+1} \mid S_t)P(X_t \mid S_t)$$

The model is parameterized by:

- **Initial state distribution:** $\boldsymbol{\pi}_i = P(S_1 = i)$
- **Transition matrix:** $A_{ij} = P(S_{t+1} = j \mid S_t = i)$
- **Observation model:**
  - For continuous observations: $A_j(\mathbf{x}) = p(\mathbf{x}_t \mid S_t = j)$

– For discrete observations: $A_{jk} = P(X_t = k \mid S_t = j)$

Important note: Although the hidden state sequence is first-order Markov, the marginal distribution of observations $X_{1:T}$ may not be Markov of any order, since states are hidden.

17.4 Three Fundamental HMM Inference Problems

Definition 17.4 (HMM Inference Problems)**.** Three key inference tasks in HMMs:

1. **Filtering:** Estimate the current state given all observations up to the present:

   $$P(Z_t \mid X_{1:t})$$

   Used for online prediction and state tracking.

2. **Smoothing:** Estimate past states given all observations:

   $$P(Z_t \mid X_{1:T}) \quad \text{where } t < T$$

   Provides better estimates than filtering since it uses future information.

3. **Prediction:** Estimate future states:

   $$P(Z_{t+k} \mid X_{1:t}) \quad \text{for } k > 0$$

   Predicts the system's future behavior.

17.5 Forward Recursion and Recursive Inference

Theorem 17.1 (Forward Recursion for HMMs)**.** Filtering in HMMs can be computed recursively using the forward algorithm:

$$P(Z_t \mid X_{1:t}) \propto p(X_t \mid Z_t) \int P(Z_t \mid Z_{t-1}) P(Z_{t-1} \mid X_{1:t-1}) dZ_{t-1}$$

This recursion alternates between:

1. **Prediction:** Propagate beliefs forward using the transition model
2. **Update/Measurement:** Correct beliefs using the new observation

For discrete HMMs with finite state spaces, this can be written using joint distributions:

$$P(Z_t, X_{1:t}) = p(X_t \mid Z_t) \int P(Z_t \mid Z_{t-1}) P(Z_{t-1}, X_{1:t-1}) dZ_{t-1}$$

The normalization is then performed to get the conditional distribution.

Remark 17.1 (Computational Efficiency)**.** For discrete HMMs with M states and T observations:

- Naive inference: $\mathcal{O}(M^T)$ (exponential in sequence length)
- Forward/backward algorithms: $\mathcal{O}(M^2 T)$ (linear in sequence length with quadratic dependence on state dimension)
- The recursive structure enables efficient dynamic programming

17.6 Inference in Linear Gaussian State Space Models

17.6.1 Conditional Gaussian Properties

Theorem 17.2 (Conditional Distribution of Joint Gaussians)**.** If two random variables are jointly Gaussian:

$$\begin{bmatrix}\mathbf{X}_A\\ \mathbf{X}_B\end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix}\boldsymbol{\mu}_A\\ \boldsymbol{\mu}_B\end{bmatrix}, \begin{bmatrix}\Sigma_A & \Sigma_{AB}\\ \Sigma_{BA} & \Sigma_B\end{bmatrix}\right)$$

Then the conditional distribution of $\mathbf{X}_A$ given $\mathbf{X}_B$ is also Gaussian:

$$\mathbb{E}[\mathbf{X}_A \mid \mathbf{X}_B = \mathbf{x}_B] = \boldsymbol{\mu}_A + \Sigma_{AB}\Sigma_B^{-1}(\mathbf{x}_B - \boldsymbol{\mu}_B)$$

$$\mathrm{Cov}(\mathbf{X}_A \mid \mathbf{X}_B = \mathbf{x}_B) = \Sigma_A - \Sigma_{AB}\Sigma_B^{-1}\Sigma_{BA}$$

This property is fundamental for Kalman filtering and other inference in Gaussian state space models.

17.6.2 Kalman Filter Algorithm

Definition 17.5 (Kalman Filter)**.** The Kalman Filter solves the filtering problem (computing $p(Z_t \mid X_{1:t})$) for linear Gaussian SSMs in closed form.

For the model:

$$\mathbf{z}_1 \sim \mathcal{N}(\boldsymbol{\mu}_0, \Omega_0) \tag{6}$$
$$\mathbf{z}_{t+1} \sim \mathcal{N}(\mathbf{A}\mathbf{z}_t, \Omega) \tag{7}$$
$$\mathbf{x}_t \mid \mathbf{z}_t \sim \mathcal{N}(\mathbf{C}\mathbf{z}_t, \mathbf{R}) \tag{8}$$

The algorithm maintains and updates:

- Filtered mean: $\hat{\mathbf{z}}_t = \mathbb{E}[\mathbf{z}_t \mid X_{1:t}]$
- Filtered covariance: $\hat{V}_t = \mathrm{Cov}(\mathbf{z}_t \mid X_{1:t})$

Prediction step:

$$\hat{\mathbf{z}}_t^{\text{pred}} = \mathbb{E}[\mathbf{z}_t \mid X_{1:t-1}] = \mathbf{A}\hat{\mathbf{z}}_{t-1} \tag{9}$$
$$\hat{V}_t^{\text{pred}} = \mathbb{V}[\mathbf{z}_t \mid X_{1:t-1}] = \mathbf{A}\hat{V}_{t-1}\mathbf{A}^T + \Omega \tag{10}$$

Kalman Gain:

$$\mathbf{K}_t = \mathrm{Cov}(\mathbf{z}_t, \mathbf{x}_t)\mathrm{Var}(\mathbf{x}_t)^{-1} = \hat{V}_t^{\text{pred}}\mathbf{C}^T(\mathbf{C}\hat{V}_t^{\text{pred}}\mathbf{C}^T + \mathbf{R})^{-1}$$

Update step:

$$\hat{\mathbf{z}}_t = \hat{\mathbf{z}}_t^{\text{pred}} + \mathbf{K}_t(\mathbf{x}_t - \mathbf{C}\hat{\mathbf{z}}_t^{\text{pred}})$$
$$\hat{V}_t = \hat{V}_t^{\text{pred}} - \mathbf{K}_t\mathbf{C}\hat{V}_t^{\text{pred}}$$

The Kalman Filter provides optimal (minimum variance) estimates when the model is Gaussian and linear.

17.6.3 Bayesian Smoothing: Forward-Backward Algorithm

Definition 17.6 (Forward-Backward Algorithm)**.** Smoothing computes $P(Z_t \mid X_{1:T})$ by combining information from past and future observations.

Define:

- **Forward message:** $\alpha_t(z_t) = P(X_{1:t}, Z_t \mid \theta)$ — evidence from past
- **Backward message:** $\beta_t(z_t) = P(X_{t+1:T} \mid Z_t, \theta)$ — evidence from future

Then the posterior can be written as:

$$P(Z_t \mid X_{1:T}) \propto \alpha_t(Z_t)\beta_t(Z_t)$$

Forward recursion:

$$\alpha_{t+1}(z_{t+1}) = P(X_{t+1} \mid z_{t+1}) \sum_{z_t} P(z_{t+1} \mid z_t)\alpha_t(z_t)$$

Backward recursion:

$$\beta_t(z_t) = \sum_{z_{t+1}} P(X_{t+1} \mid z_{t+1})P(z_{t+1} \mid z_t)\beta_{t+1}(z_{t+1})$$

This algorithm is also known as the **Baum-Welch algorithm** in the HMM literature.

17.6.4 Viterbi Algorithm: Maximum Most Likely Path

Definition 17.7 (Viterbi Algorithm)**.** The Viterbi algorithm finds the most likely sequence of hidden states:

$$S^* = \arg\max_{S_{1:T}} P(S_{1:T} \mid X_{1:T}, \theta)$$

Instead of summing over paths (as in forward-backward), it tracks the maximum:

$$\alpha_t^{\max}(j) = \max_i \alpha_{t-1}^{\max}(i) A_{ij} p(X_t \mid j)$$

The algorithm replaces the sum-product computations with max-sum, reducing complexity while finding the Viterbi path (most likely hidden state sequence).

This is useful for applications like speech recognition where we need the single most likely sequence rather than marginal posteriors.

17.6.5 Kalman Smoothing

Definition 17.8 (Kalman Smoothing)**.** Kalman smoothing extends the Kalman filter to the smoothing problem: computing $p(Z_t \mid X_{1:T})$ for $t \leq T$ in linear Gaussian models.

Starting with forward-filtered estimates $\hat{\mathbf{z}}_t, \hat{V}_t$, we can compute smoothed posteriors by a backward pass.

Smoothing gain:

$$\mathbf{J}_t = \mathrm{Cov}(\mathbf{z}_t, \mathbf{z}_{t+1} \mid X_{1:t})\mathrm{Var}(\mathbf{z}_{t+1} \mid X_{1:t})^{-1} = \hat{V}_t \mathbf{A}^T (\hat{V}_{t+1}^{\mathrm{pred}})^{-1}$$

Smoothed mean:

$$\bar{\mathbf{z}}_t = \hat{\mathbf{z}}_t + \mathbf{J}_t(\bar{\mathbf{z}}_{t+1} - \mathbf{A}\hat{\mathbf{z}}_t)$$

Smoothed covariance:

$$\bar{V}_t = \hat{V}_t + \mathbf{J}_t(\bar{V}_{t+1} - \hat{V}_{t+1}^{\text{pred}})\mathbf{J}_t^T$$

Starting from the final filtered estimate at $t = T$, we recursively compute smoothed estimates backward in time.

Key advantage: Smoothing provides better state estimates than filtering because it uses all observations, not just those up to the current time.

18 Parameter Learning in State Space Models

18.1 ML Learning for Linear Gaussian State Space Models

Definition 18.1 (SSM Parameter Learning)**.** For the linear Gaussian SSM with parameters $\boldsymbol{\theta} = \{\boldsymbol{\mu}_0, \Omega_0, \mathbf{A}, \Omega, \mathbf{C}, \mathbf{R}\}$, we can use EM to learn these parameters.

The key observation is that the measurement model depends only on the observation $\mathbf{x}_t$ and latent state $\mathbf{z}_t$. In the M-step, the derivatives of the free energy with respect to parameters can be expressed in terms of expected sufficient statistics.

For the observation matrix $\mathbf{C}$:

$$\mathbf{C}_{\text{new}} = \left(\sum_{i,t} \mathbf{x}_t \langle \mathbf{z}_t \rangle^T \right) \left(\sum_{i,t} \langle \mathbf{z}_t \mathbf{z}_t^T \rangle \right)^{-1}$$

For the observation noise covariance:

$$\mathbf{R}_{\text{new}} = \frac{1}{nT} \left(\sum_{i,t} \mathbf{x}_t \mathbf{x}_t^T - \mathbf{C} \sum_{i,t} \langle \mathbf{z}_t \rangle \mathbf{x}_t^T \right)$$

For the transition matrix $\mathbf{A}$:

$$\mathbf{A}_{\text{new}} = \left(\sum_{i,t} \langle \mathbf{z}_{t+1} \mathbf{z}_t^T \rangle \right) \left(\sum_{i,t} \langle \mathbf{z}_t \mathbf{z}_t^T \rangle \right)^{-1}$$

For the process noise covariance:

$$\Omega_{\text{new}} = \frac{1}{nT} \left(\sum_{i,t} \langle \mathbf{z}_{t+1} \mathbf{z}_{t+1}^T \rangle - \mathbf{A} \sum_{i,t} \langle \mathbf{z}_{t+1} \mathbf{z}_t^T \rangle \right)$$

The E-step must compute the expected sufficient statistics: $\sum_t \langle \mathbf{z}_t \rangle$, $\sum_t \langle \mathbf{z}_t \mathbf{z}_t^T \rangle$, and $\sum_t \langle \mathbf{z}_{t+1} \mathbf{z}_t \rangle$ using the Kalman smoother.

18.2 Online Gradient-Based Learning

Definition 18.2 (Online Learning for SSMs)**.** For streaming applications, we can use online gradient descent on the log-likelihood:

$$\ell(\boldsymbol{\theta}) = \sum_t \log p(\mathbf{x}_t \mid \mathbf{x}_{1:t-1}) = \sum_t \ell_t$$

For each observation, the single-step likelihood is:

$$\ell_t = -\frac{1}{2} \log |\Sigma_t| - \frac{1}{2} (\mathbf{x}_t - \mathbf{C}\hat{\mathbf{z}}_t^{\text{pred}})^T \Sigma_t^{-1} (\mathbf{x}_t - \mathbf{C}\hat{\mathbf{z}}_t^{\text{pred}})$$

where:

- $\hat{\mathbf{z}}_t^{\text{pred}}$ is the predicted state from the Kalman filter
- $\Sigma_t = \mathbf{C} V_t^{\text{pred}} \mathbf{C}^T + \mathbf{R}$ is the predicted observation variance

We compute the gradient $\nabla_{\boldsymbol{\theta}} \ell_t$ and perform stochastic gradient updates:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \nabla_{\boldsymbol{\theta}} \ell_t$$

This approach is useful for online adaptation and for very long sequences where batch EM is intractable.

18.3 EM Learning for Hidden Markov Models

Definition 18.3 (HMM Parameter Learning with EM (Baum-Welch))**.** For HMMs with discrete observations, the EM algorithm is known as the Baum-Welch algorithm. The M-step updates are:

Initial state distribution:

$$\pi_i^{\text{new}} = \frac{1}{n} \sum_i \gamma_1(i)$$

Transition probabilities:

$$A_{ij}^{\text{new}} = \frac{\sum_t \xi_t(i \to j)}{\sum_j \sum_t \xi_t(i \to j)} = \frac{\text{expected transitions } i \to j}{\text{expected times in state } i}$$

where $\xi_t(i \to j) = P(S_t = i, S_{t+1} = j \mid X_{1:T})$ are the expected bigram counts.

Observation distribution: For discrete observations:

$$P(X_t = k \mid S_t = j)^{\text{new}} = \frac{\sum_t \mathbb{1}[X_t = k]\gamma_t(j)}{\sum_t \gamma_t(j)}$$

For Gaussian emissions:

$$\boldsymbol{\mu}_j^{\text{new}} = \frac{\sum_t \gamma_t(j)\mathbf{x}_t}{\sum_t \gamma_t(j)}$$

$$\Sigma_j^{\text{new}} = \frac{\sum_t \gamma_t(j)(\mathbf{x}_t - \boldsymbol{\mu}_j^{\text{new}})(\mathbf{x}_t - \boldsymbol{\mu}_j^{\text{new}})^T}{\sum_t \gamma_t(j)}$$

These updates use the posterior marginals $\gamma_t(j) = P(S_t = j \mid X_{1:T})$ computed in the E-step using the forward-backward algorithm.

Remark 18.1. Baum-Welch as EM The Baum-Welch algorithm is exactly the EM algorithm applied to HMMs:

- **E-step:** Use the forward-backward algorithm to compute $\gamma_t(j)$ and $\xi_t(i \to j)$
- **M-step:** Update parameters using the expected counts derived from these posterior marginals
- The algorithm is guaranteed to increase (or maintain) the likelihood at each iteration

19 Markov Chain Theory and Sampling Methods

Markov chains are fundamental to understanding both sequential systems and modern sampling algorithms. A Markov chain models systems where the future depends only on the present state, not on history. Beyond their intrinsic importance for sequential modeling, Markov chain theory underpins Markov Chain Monte Carlo (MCMC) methods—powerful algorithms for sampling from complex, high-dimensional distributions. This section develops the theory of Markov chains and then shows how this theory justifies sampling algorithms essential for Bayesian inference.

19.1 Markov Chain Basics and Classification

Definition 19.1 (Markov Chain Properties)**.** A finite-state Markov chain is characterized by its transition matrix $\mathbf{P}$ where P_{ij} is the probability of transitioning from state i to state j.

Key properties:

1. **Accessibility:** State j is accessible from state i (denoted $i \to j$) if $P_{ij}^{(n)} > 0$ for some $n \geq 0$
2. **Communication:** States i and j communicate ($i \leftrightarrow j$) if they are mutually accessible
3. **Irreducibility:** A Markov chain is irreducible if all states communicate (there is only one communicating class)
4. **Recurrence:** A state is recurrent if, starting from it, the chain will eventually return to it with probability 1. Otherwise, it is transient

19.2 Periodicity and Ergodicity

Definition 19.2 (Periodicity)**.** The period of state i is:

$$d(i) = \gcd\{n \geq 1 : P_{ii}^{(n)} > 0\}$$

A Markov chain is **aperiodic** if $d(i) = 1$ for all states i.

Definition 19.3 (Ergodic States)**.** A state that is positive recurrent and aperiodic is called **ergodic**.

Key properties:

- **Mean recurrence time:** $M_i = \mathbb{E}[R_i \mid X_0 = i] = \sum_{k=1}^{\infty} k f_{ii}^{(k)}$, where $f_{ii}^{(k)}$ is the probability of first return to i at time k
- **Positive recurrence:** $M_i < \infty$
- **Null recurrence:** $M_i = \infty$

19.3 Stationary Distribution and Convergence

Theorem 19.1 (Basic Limit Theorem for Markov Chains)**.** For an irreducible, aperiodic, and positive recurrent Markov chain:

$$\lim_{n\to\infty} P_{ij}^{(n)} = \pi_j = \frac{1}{M_j}$$

exists and is independent of the initial state i.
The stationary distribution $\boldsymbol{\pi}$ satisfies:

- $\mathbf{P}^T\boldsymbol{\pi} = \boldsymbol{\pi}$ (it is an eigenvector of $\mathbf{P}^T$ with eigenvalue 1)
- π_j is the long-run fraction of time the chain spends in state j

Remark 19.1 (Convergence Rate)**.** The rate of convergence to the stationary distribution depends on the spectral gap—the ratio between the largest and second-largest eigenvalues of $\mathbf{P}$. Chains with larger spectral gaps mix faster.

20 Sampling Methods

20.1 Direct Sampling from Mixture Distributions

Definition 20.1 (Marginalized Posterior Predictive)**.** When performing Bayesian inference with latent variables, we can integrate out the uncertainty in parameters:

$$p(X_{n+1} \mid X_{1:n}) = \int p(X_{n+1} \mid \theta)p(\theta \mid X_{1:n})d\theta = \mathbb{E}_{p(\theta|X_{1:n})}[p(X_{n+1} \mid \theta)]$$

This mixture distribution can be approximated by:

$$p(X_{n+1} \mid X_{1:n}) \approx \frac{1}{m}\sum_{i=1}^{m} p(X_{n+1} \mid \theta_i)$$

where θ_i are samples from the posterior.

20.2 Rejection Sampling

Definition 20.2 (Rejection Sampling)**.** To sample from a target distribution $p(x)$, we use a proposal distribution $r(x)$ (called an envelope) such that $p(x) \le M \cdot r(x)$ for some $M > 0$.

Algorithm:

1. Sample $X_i \sim r(x)$
2. Sample $Y_i \sim \text{Uniform}(0, r(X_i))$
3. If $Y_i < p(X_i)$, keep X_i; otherwise discard it

Advantage: We don't need to know the normalizing constant of p.
Disadvantage: The acceptance rate is $\frac{1}{M}$, which is low if M is large.

20.3 Importance Sampling

Definition 20.3 (Importance Sampling)**.** Importance sampling computes expectations under a target distribution p using samples from a proposal distribution q:

$$\mathbb{E}_p[f(x)] = \int f(x)p(x)dx = \int f(x)\frac{p(x)}{q(x)}q(x)dx = \mathbb{E}_q\left[f(x)\frac{p(x)}{q(x)}\right]$$

If we have i.i.d. samples $X_1, \ldots, X_n \sim q$, we can approximate:

$$\mathbb{E}_p[f(x)] \approx \frac{1}{n}\sum_{i=1}^{n} f(x_i)\frac{p(x_i)}{q(x_i)} = \frac{1}{n}\sum_{i=1}^{n} f(x_i)w_i$$

where $w_i = \frac{p(x_i)}{q(x_i)}$ are the importance weights.

21 Markov Random Fields and Graphical Models

Graphical models provide a powerful framework for representing complex dependencies in high-dimensional distributions through graphs, where nodes are variables and edges represent conditional dependencies. Markov Random Fields are a natural way to encode local structure in joint distributions, and energy-based models like the Ising and Potts models show how to design priors that capture spatial coherence and smoothness. This section develops these ideas and shows how Markov Random Fields enable both tractable inference and principled model design.

21.1 Neighbourhood Graphs and Random Fields

Definition 21.1 (Neighbourhood Graph)**.** A neighbourhood graph $\mathcal{N}(\mathcal{V}_r, \mathcal{W}_r)$ consists of:

- $\mathcal{V}_r$: the vertex set
- $\mathcal{W}_r$: the set of edge weights

This defines a **random field**, a collection of random variables indexed by vertices.

Definition 21.2 (Neighbourhood of a Vertex)**.** The neighbourhood of vertex i is the set:

$$\partial(i) = \{j \mid W_{ij} \neq 0\}$$

The **Markov blanket** of vertex θ_i is:

$$\mathrm{MB}(\theta_i) = \{\partial_j : j \in \partial(i)\}$$

Definition 21.3 (Markov Property)**.** A random field has the **Markov property** if each variable is conditionally independent of all other variables given its Markov blanket:

$$p(\theta_i \mid \boldsymbol{\theta}_{\setminus i}) = p(\theta_i \mid \boldsymbol{\theta}_{\partial i})$$

where $\boldsymbol{\theta}_{\partial i}$ denotes the variables in the Markov blanket of θ_i.

Definition 21.4 (Markov Random Field)**.** A random field with the Markov property is called a **Markov Random Field (MRF)**.

21.2 Energy Functions and Exponential Family Connection

Definition 21.5 (Energy Functions)**.** Any strictly positive probability or density p can be written in the form:

$$p(x) = \frac{1}{Z} \exp(-H(x))$$

where:

- $H : \mathcal{X} \to \mathbb{R}$ is the **energy function** (also called **potential** or **cost function**)
- $Z = \int p(x) \exp(-H(x))\, dx$ is the **partition function**

If H is linear in the parameters, we have an exponential family distribution.

21.3 The Potts and Ising Models

Definition 21.6 (The Potts Model)**.** The Potts model is defined as:

$$p(\theta_1, \ldots, \theta_n) = \frac{1}{Z} \exp\left(\sum_{ij} w_{ij} \mathbb{1}[\theta_i = \theta_j]\right)$$

where the sum is over pairs of adjacent vertices, and w_{ij} are the edge weights.

Definition 21.7 (The Ising Model)**.** The Ising model is a Potts model with binary variables $\theta_i \in \{-1, +1\}$ and weights $w_{ij} \in \{0, 1\}$:

$$p(\theta_1, \ldots, \theta_n) = \frac{1}{Z(\beta)} \exp\left(\beta \sum_{ij} \mathbb{1}[\theta_i = \theta_j]\right)$$

Remark 21.1 (Parameter Interpretation in Potts Model)**.** In the Potts model, the ratio:

$$\frac{p(\theta_i = \beta_j)}{p(\theta_i \neq \beta_j)} = \exp(\beta w_{ij})$$

- When $w_{ij} > 0$: leads to ≥ 1, preferring equal states (smoothness)
- When $w_{ij} < 0$: leads to < 1

Remark 21.2 (Inverse Temperature)**.** The parameter β is the **inverse temperature**:

- As $\beta \uparrow$: the model becomes "colder", more deterministic, and more smoothed. Very heavy penalty on boundary violations. Boundaries only appear where forced by conflicting data or very negative weights.
- As $\beta \downarrow$: the model becomes "hotter", more random. Weaker coupling, flatter distribution, closer to uniform.

21.4 Spatial Models and Smoothing

Definition 21.8 (Spatial Model)**.** A spatial model encodes each observation X_i through a likelihood $\mathcal{L}(x_i|\theta_i)$ with prior $P(\theta_i)$ coming from an MRF:

$\mathcal{L}(x_i|\theta_i)$ is like emission probability, similar to an HMM

Remark 21.3 (Spatial Smoothing with MRF)**.** For positive weights, an MRF encourages the model to explain neighbouring observations X_i and X_j using the same parameter values.

21.5 Bayesian Mixture Models

Definition 21.9 (Bayesian Mixture Model)**.** A Bayesian mixture model is defined as:

$$P(x) = \sum_{k=1}^{K} c_k p(x|\theta_k)$$

where:

- $p(x|\theta_k)$ is an exponential family distribution
- $(\theta_1, \ldots, \theta_K)$ are drawn i.i.d. from a prior distribution $P(\theta_k)$
- $c_1, \ldots, c_K$ are sampled from a K-dimensional Dirichlet distribution

21.6 Markov Chains and MCMC

Definition 21.10 (Continuous Markov Chain)**.** A continuous Markov chain is defined by:

- An initial distribution P_{init}
- A conditional probability (transition probability or transition kernel) $p(x|y)$

Definition 21.11 (Invariant Distribution in Continuous MC)**.** A distribution is **invariant** for a Markov chain if:

$$\int \mu_y(y) P_{\mu,\nu}(x)\, dx = P_{\mu,\nu}(y)$$

This is the continuous analogue of the discrete case: $\sum_j p_j (Pm\nu)_j = (Pm\nu)_j$

Definition 21.12 (Detailed Balance)**.** A transition kernel $T(x \to x')$ satisfies **detailed balance** with respect to target distribution $\pi(x)$ if:

$$\pi(x)T(x \to x') = \pi(x')T(x' \to x), \quad \forall \text{ pair } (x, x')$$

This implies stationarity: $\int \pi(x)T(x \to x')\, dx = \pi(x')$

21.7 Metropolis-Hastings Algorithm

Definition 21.13 (Metropolis-Hastings Algorithm)**.** The Metropolis-Hastings algorithm proposes a candidate $x' \sim g(x \to x')$ and accepts it with probability:

$$A(x \to x') = \min\left(1, \frac{\pi(x')g(x' \to x)}{\pi(x)g(x \to x')}\right)$$

The resulting transition kernel is:

$$T(x' \to x) = g(x' \to x)A(x' \to x)$$

To implement this:

1. Sample a candidate $X^* \sim g(\cdot \mid x_t)$ from the proposal
2. Compute the acceptance ratio: $A(x_t \to x^*)$
3. Draw $u \sim \text{Unif}(0, 1)$
4. If $u \leq A(x_t \to x^*)$, accept and set $X_{t+1} = x^*$
5. Otherwise, keep $X_{t+1} = X_t$ (reject the proposal)

22 Advanced MCMC and Optimization Methods

With the Markov chain foundations established, we now develop practical MCMC algorithms (Gibbs sampling and Metropolis-Hastings) for inference in complex probabilistic models. We also cover optimization methods that complement sampling approaches. These techniques—including stochastic gradient descent, Newton's method, and interior point methods—form the computational core of modern unsupervised learning.

22.1 Stochastic Hill Climbing

Remark 22.1 (Stochastic Hill Climbing)**.** In Metropolis-Hastings, we accept candidate x' if:

$$\pi(x')g(x' \to x) > \pi(x)g(x \to x')$$

- We always accept the proposed value X_{i+1} if it increases probability under π
- If it decreases the probability, we accept it with some probability
- This is gradient ascent on π, in the direction of increasing probability p
- It still maintains exploration, avoiding trapping in local maxima

22.2 Global Balance and Convergence

Definition 22.1 (Global Balance)**.** The condition:

$$\sum_j \pi(x)T(x \to x') = \pi(x')$$

is called **global balance**. Detailed balance is a stronger, pairwise version.

22.3 Burn-in and Mixing Time

Definition 22.2 (Burn-in and Mixing Time)**.** The number m of steps required until P_m (the m-step distribution) converges to π is called the **mixing time**. The first m samples are called the **burn-in phase** or **discard phase**.

Remark 22.2 (Autocorrelation Function)**.** Use the autocorrelation function to check after burn-in. If the autocorrelation is close to 0, create i.i.d samples by picking every L steps (where autocorrelation is close to 0).

22.4 Approximation by Gaussian Distribution

Remark 22.3 (Unimodal Approximation)**.** If π is unimodal and can be roughly approximated by a Gaussian, then Var$[p]$ should be chosen as the smallest covariance component of π.

22.5 Full Conditional Distributions

Definition 22.3 (Full Conditional Distribution)**.** The full conditional distribution of X_d is:

$$p(x_d|x_1, \ldots, x_{d-1}, x_{d+1}, \ldots, x_D), \text{ given } P(X) \text{ a distribution on } \mathbb{R}^D$$

22.6 Gibbs Sampler

Definition 22.4 (Gibbs Sampler)**.** Given a vector X_i, we generate X_{i+1} as follows:

$$X_{i+1,1} \sim P(\cdot|X_{i,2}, \ldots, X_{i,D})$$

$$X_{i+1,d} \sim P(\cdot|X_{i,1}, \ldots, X_{i,d-1}, X_{i+1,d+1}, \ldots, X_{i,D})$$

$$X_{i+1,D} \sim P(\cdot | X_{i+1,1}, \ldots, X_{i+1,D-1})$$

For each dimension $d \in \{1, \ldots, D\}$, the variables $X_{1,d}, X_{2,d}, \ldots$ form again a Markov process and it is an MH sampler. So Gibbs with D full conditionals is a family of coupled MH samplers.

These MH samplers all have **acceptance probability 1**.

Remark 22.4 (Why Gibbs Sampling has Acceptance Probability 1)**.** In Gibbs sampling, each step is a Metropolis-Hastings update with the full conditional distribution as the proposal. The acceptance ratio simplifies nicely:

$$A(x \to x') = \min\left(1, \frac{\pi(x')q(x \mid x')}{\pi(x)q(x' \mid x)}\right)$$

where $q(x' \mid x) = p(x'_d \mid x_{\setminus d})$ and $q(x \mid x') = p(x_d \mid x_{\setminus d})$.

Both $\pi(x)$ and $\pi(x')$ factor as $\pi(x_d|x_{\setminus d})p(x_{\setminus d})$, andthe proposal ratio becomes:

$$\frac{\pi(x')q(x \mid x')}{\pi(x)q(x' \mid x)} = \frac{p(x'_d|x_{\setminus d})p(x_{\setminus d}) \cdot p(x_d|x_{\setminus d})}{p(x_d|x_{\setminus d})p(x_{\setminus d}) \cdot p(x'_d|x_{\setminus d})} = 1$$

Thus, every proposal is accepted with probability 1, making Gibbs sampling a **deterministic updating rule** at each coordinate given the current state of other coordinates.

Definition 22.5 (Gibbs on Ising Model)**.** For the Ising model:

$$\hat{p}(\theta_d|\theta_1, \ldots, \theta_{d-1}, \theta_{d+1}, \ldots, \theta_D) = \hat{p}(\theta_d|\theta_{\text{left}}, \theta_{\text{right}}, \theta_{\text{up}}, \theta_{\text{down}})$$

$$= \exp\left(\beta\left(1[\theta_d = \theta_{\text{left}}] + 1[\theta_d = \theta_{\text{right}}] + 1[\theta_d = \theta_{\text{up}}] + 1[\theta_d = \theta_{\text{down}}]\right)\right)$$

22.7 Gradient-Based Methods

Definition 22.6 (Gradient of a Scalar Function)**.** The gradient $\nabla f(x)$ of $f : \mathbb{R}^d \to \mathbb{R}$ at a point $x \in \mathbb{R}^d$ is a vector in the domain $\mathbb{R}^d$ in the direction in which f increases most rapidly at x.

Remark 22.5 (Orthogonality to Contour Lines)**.** The gradient is orthogonal to contour lines.

22.8 Newton's Method

Definition 22.7 (Newton's Method)**.** Newton's method minimizes with updates:

$$X_{i+1} = X_i - \frac{\nabla f(x_i)}{f'(x_i)}$$

With alternative formulation:

$$X_{i+1} = X_i - f'(x_i)^{-1}\nabla f(x_i), \quad \text{or} \quad X_{i+1} = X_i - H^{-1}(x_i)\nabla f(x_i)$$

22.9 Barrier Function and Interior Point Methods

Definition 22.8 (Barrier Function)**.** A barrier function t approximates the constraint by a smoothing function, e.g., $g_t(x) = -\frac{1}{t}\log(-x)$

Remark 22.6 (Constrained Optimization with Barrier Function)**.** Interior point methods approximately solve $\min f(x)$ s.t. $g_i(x) \leq 0 \Rightarrow \min(f(x) + \sum \beta_i g_i(x))$

Convergence conditions: Robbins-Monro conditions

$$\sum_{i=1}^{\infty} \alpha_i(t) \to \infty, \quad \sum_{i=1}^{\infty} \alpha_i(t)^2 < \infty$$

Points arbitrarily far from y_0 are reachable. Also required in ordinary gradient descent.

22.10 Stochastic Gradient Descent

Definition 22.9 (Stochastic Gradient Descent)**.** Stochastic gradient descent uses:

$$\hat{x}_{i+1} = \hat{x}_i - \alpha(n)\nabla\hat{f}(x_i)$$

with $\nabla\hat{f}(x) = \nabla f(x) + \varepsilon$, where $\mathbb{E}[\varepsilon] = 0$.

Convergence conditions: Robbins-Monro conditions

$$\sum_{i=1}^{\infty} \alpha_i(t) \to \infty, \quad \frac{\text{finite variance}}{(\text{Var}(\alpha_n)\hat{x}_i)^2} \text{ and } \quad \text{Var}\left[\sum_{i=1}^{\infty} \alpha_i(n)\varepsilon_i\right] = \left(\sum_{n=1}^{\infty} \alpha_n^2\right)^2$$